Java OOP Interview Preparation Notes

(With Tricky Scenarios & Gotchas)

---

1. The Four Pillars — Quick Recap

Pillar	One-line definition
Encapsulation	Bundling data + methods, restricting direct access via access modifiers
Abstraction	Hiding implementation details, exposing only "what," not "how"
Inheritance	Acquiring properties/behavior of a parent class
Polymorphism	One interface, many implementations (compile-time & runtime)

Tricky question: *"Give an example where encapsulation is broken even with private fields."*
Answer: If a private field is a mutable object (e.g., `Date`, `List`) and you return it directly via a getter without defensive copying, the caller can modify internal state.

```
private List<String> items = new ArrayList<>();
public List<String> getItems() { return items; } // BROKEN — caller can mutate
public List<String> getItems() { return new ArrayList<>(items); } // FIXED
```

---

2. Overloading vs Overriding — Where People Trip Up

Aspect	Overloading	Overriding
Binding	Compile-time (static)	Runtime (dynamic)
Return type	Can differ	Must be same or covariant
Access modifier	Can differ freely	Cannot be more restrictive
Static methods	Can be overloaded	**Cannot be overridden** (only hidden)

Aspect	Overloading	Overriding
Private methods	Can be overloaded	**Cannot be overridden** (not inherited)
`final` methods	Can be overloaded	Cannot be overridden
Exceptions	No restriction	Can't throw broader **checked** exceptions

Tricky scenario — Static method "hiding" vs overriding:

```
class Parent {
    static void greet() { System.out.println("Parent"); }
}
class Child extends Parent {
    static void greet() { System.out.println("Child"); }
}
Parent p = new Child();
p.greet(); // prints "Parent" — static methods resolved by REFERENCE TYPE, not object
```

This is the #1 gotcha interviewers use to check if you actually understand static vs dynamic binding.

Tricky scenario — Overloading ambiguity with autoboxing/varargs:

```
void test(int a) {}
void test(Integer a) {}
void test(int... a) {}
test(5); // calls test(int) — most specific match wins:
         // exact match > widening > autoboxing > varargs
```

Tricky scenario — Overriding and access modifiers:

```
class Parent {
    protected void show() {}
}
class Child extends Parent {
    public void show() {} // OK — widening access is allowed
    // private void show() {} // COMPILE ERROR — cannot reduce visibility
}
```

Tricky scenario — Covariant return types:

```
class Animal {
    Animal reproduce() { return new Animal(); }
}
```

```
class Dog extends Animal {
    @Override
    Dog reproduce() { return new Dog(); } // Legal! Return type is a subtype
}
```

---

3. Constructors — Common Trap Questions

- **Constructors are NOT inherited.** Only `super()` chaining gives access.
- If you don't write any constructor, compiler adds a **no-arg default constructor**. The moment you write ANY constructor, the default one disappears.
- `this()` and `super()` must be the **first statement** — and you can never call both in the same constructor.

Tricky scenario — Constructor call order:

```
class A {
    A() { System.out.println("A"); method(); }
    void method() { System.out.println("A.method"); }
}
class B extends A {
    int x = 10;
    B() { System.out.println("B, x=" + x); }
    @Override
    void method() { System.out.println("B.method, x=" + x); }
}
new B();
```

Output:

```
A
B.method, x=0    <-- x is NOT yet initialized! (field initializers run after super())
B, x=10
```

This is a **classic trap**: calling an overridable method from a constructor is dangerous because the subclass override may run before subclass fields are initialized.

---

4. Abstract Class vs Interface — The "Why Both Exist" Question

Feature	Abstract Class	Interface
Multiple inheritance	No (single class inheritance)	Yes (implement multiple)
Constructors	Yes	No
State (instance fields)	Yes	No (only `static final` constants)
Access modifiers on methods	Any	`public` by default (Java 9+ allows `private`)
Default/static methods	Always could have body	Since Java 8: `default` and `static` methods allowed

Tricky scenario — Diamond Problem with default methods:

```
interface A { default void hello() { System.out.println("A"); } }
interface B { default void hello() { System.out.println("B"); } }
class C implements A, B {
    // MUST override — compiler forces you to resolve ambiguity
    public void hello() {
        A.super.hello(); // explicitly choosing which interface's default to call
    }
}
```

Interviewers ask this to see if you know Java **avoids** true diamond inheritance ambiguity by forcing explicit resolution — it doesn't silently pick one.

When to choose abstract class over interface: when you need shared state/fields, constructors, or non-public helper methods — otherwise prefer interfaces for flexibility (favor composition/interfaces per "program to an interface" principle).

---

5. Polymorphism — Deeper Gotchas

Tricky scenario — Object slicing doesn't exist in Java (unlike C++), but reference-type behavior confuses people:

```
class Animal { String sound() { return "..."; } }
class Cat extends Animal { String sound() { return "Meow"; } }

Animal a = new Cat();
```

```
System.out.println(a.sound()); // "Meow" — method calls are dynamically bound
System.out.println(a instanceof Cat); // true — object's actual type matters
```

But **field access is NOT polymorphic** — fields are resolved by reference type:

```
class Animal { String name = "Animal"; }
class Cat extends Animal { String name = "Cat"; }

Animal a = new Cat();
System.out.println(a.name); // "Animal" — fields use STATIC binding!
```

This field-vs-method binding difference is one of the most commonly asked "gotcha" questions.

Tricky scenario — Overriding and checked exceptions:

```
class Parent {
    void read() throws IOException {}
}
class Child extends Parent {
    // void read() throws Exception {} // COMPILE ERROR — broader checked exception
    void read() throws FileNotFoundException {} // OK — narrower is fine
    // void read() {} // Also OK — can throw fewer/no exceptions
}
```

---

6. equals() and hashCode() — Always Asked

The contract:

1. If `a.equals(b)` is true, `a.hashCode() == b.hashCode()` **must** be true.
2. The reverse is NOT required (hash collisions are allowed).
3. If you override `equals()`, you **must** override `hashCode()` — otherwise HashMap/HashSet break silently.

Tricky scenario:

```
class Point {
    int x, y;
    @Override
    public boolean equals(Object o) {
        if (!(o instanceof Point p)) return false;
        return x == p.x && y == p.y;
    }
    // forgot hashCode()!
```

```
}
Set<Point> set = new HashSet<>();
set.add(new Point(1,2));
System.out.println(set.contains(new Point(1,2))); // false! Different hashCode buckets
```

Also asked: `==` vs `.equals()` — `==` compares references for objects (values for primitives); `.equals()` is content comparison (if overridden).

---

7. Composition vs Inheritance

- **"Favor composition over inheritance"** — classic interview line, be ready to explain *why*:
  - Inheritance breaks encapsulation (subclass depends on parent's internal implementation — the "fragile base class" problem)
  - Composition is more flexible — behavior can change at runtime by swapping the composed object
  - Deep inheritance hierarchies become hard to maintain and test

Tricky scenario — Fragile base class problem:

```
class InstrumentedHashSet<E> extends HashSet<E> {
    int addCount = 0;
    @Override
    public boolean add(E e) {
        addCount++;
        return super.add(e);
    }
    @Override
    public boolean addAll(Collection<? extends E> c) {
        addCount += c.size();
        return super.addAll(c);
    }
}
```

Bug: `HashSet.addAll()` internally calls `add()` — so elements get double-counted! This is the textbook example (from *Effective Java*) of why extending concrete classes is risky, and why composition (wrapping instead of extending) is safer.

---

8. Static Binding vs Dynamic Binding — Summary Table

Element	Binding type
Instance method (non-private, non-final)	Dynamic (runtime)
Static method	Static (compile-time)
Private method	Static (compile-time)
Field	Static (compile-time)
final method	Static (technically eligible, but still not overridable)

9. Abstract Classes — Instantiation & Constructor Traps

- You **cannot instantiate** an abstract class, but it **can have a constructor** — called implicitly via `super()` when a subclass object is created.
- An abstract class **can exist without any abstract methods** (used to prevent instantiation while sharing common code).
- A class with even one abstract method **must** be declared abstract.

10. Interfaces — Java 8+ Nuances (frequently tested now)

```
interface Shape {
    double area();                                  // abstract
    default void describe() {                       // default (Java 8)
        System.out.println("Area: " + area());
    }
    static Shape unitCircle() {                      // static (Java 8)
        return () -> Math.PI;
    }
    private double helper() { return 0; }             // private (Java 9)
}
```

- `static` interface methods are NOT inherited by implementing classes — must be called via interface name.
- `private` interface methods exist only to avoid code duplication between default methods; not accessible outside the interface.

11. Rapid-Fire "Explain the Output" Questions to Practice

Interviewers love giving snippets and asking "what's printed?" Practice these patterns:

1. Method overriding + field hiding combined in one hierarchy
2. Constructor calling an overridden method
3. Static vs instance initializer block execution order
4. `super.method()` vs `this.method()` inside overridden methods
5. Exception thrown from a constructor — does the object get partially created?
6. Multiple levels of inheritance with `protected` fields and overriding at each level